

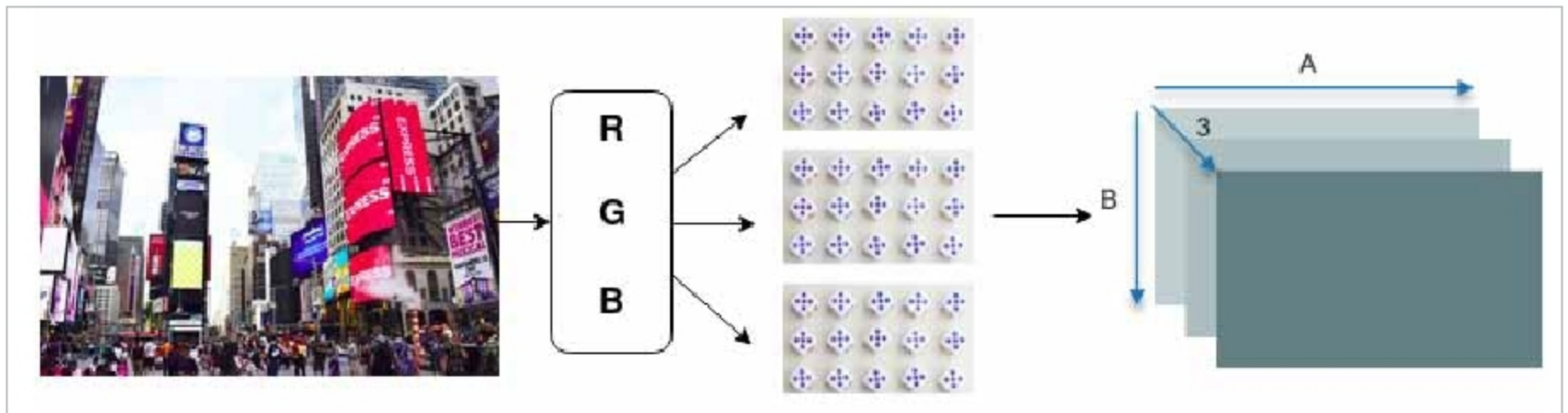


Figure 3: Conversion of an image to numpy arrays

converting them into grayscale, type:

```
import numpy as np
import cv2

img = cv2.imread('Image123.png',0)
#write the name of an image
cv2.imshow('image',img)
k = cv2.waitKey(0) & 0xFF
if k == 27:      # wait for ESC key to
    exit
    cv2.destroyAllWindows()
elif k == ord('s'): # wait for 's' key
    to save and exit
    cv2.imwrite('Firefox_
wallpapergray.png',img)
    cv2.destroyAllWindows()
```

`cv2.imread` reads the selected image, and the 0 parameter turns the image into grayscale; `cv2.imshow` shows the converted image.

Drawing and writing on an image can be done as follows:

```
import numpy as np
import cv2
img = cv2.imread('black.jpg',cv2.
IMREAD_COLOR)

cv2.line(i
mg,(0,0),(511,511),(255,0,0),5)
cv2.rectangle(i
mg,(384,0),(510,128),(0,255,0),3)
cv2.circle(img,(447,63), 63, (0,0,255),
-1)

font = cv2.FONT_HERSHEY_SIMPLEX
cv2.putText(img,'OpenCV',(10,500),
```

```
font, 4,(255,255,255),2,cv2.LINE_AA)

cv2.imshow('image',img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

`cv2.line` draws the line with the given coordinates on the image, `cv2.rectangle` and `cv2.circle` draw a rectangle and circle respectively, and `cv2.putText` writes the given text. Here, we have used the Hershey Simplex font. The output is shown in Figure 4.

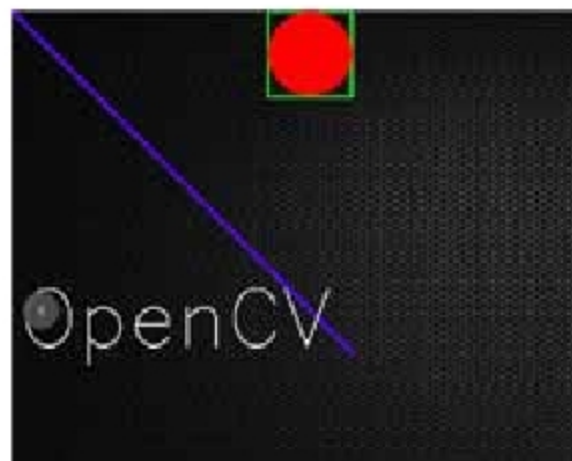


Figure 4: Output after drawing and writing on an image

Feature and template matching

Template matching is basically a part of one image matching another image. The code for this is as follows:

```
import cv2
import numpy as np
img_bgr=cv2.imread('sc1.png')
img_gray=cv2.cvtColor(img_bgr, cv2.
COLOR_BGR2GRAY)

template=cv2.imread('sc2.png',0)
w,h=template.shape[::-1]
```

```
res=cv2.matchTemplate(img_
gray,template,cv2.TM_CCOEFF_NORMED)
threshold=0.8
loc=np.where(res>=threshold)

for pt in zip(*loc[::-1]):
    cv2.rectangle(img_bgr, pt,
(pt[0]+w, pt[1]+h), (0,255,255),2)

cv2.imshow('detected',img_bgr)
cv2.waitKey()
```

`cv2.imread` reads the selected image, `cv2.cvtColor` converts the image into grayscale, `w` and `h` variables are the positions of `x` and `y`-axis, `cv2.matchTemplate` helps to match the common area of two images with threshold 80 per cent, and `cv2.rectangle` marks the area that is matching in the image.

The output is shown in Figures 5 and 6.



Figure 5: Output of template matching